# concealed_hits_triage

# BOB-205 — Triage of the four concealed §11.4.252 fail-open hits

| Field | Value |
|---|---|
| Revision | 1 |
| Created | 2026-08-26T00:00:00Z |
| Last modified | 2026-08-26T00:00:00Z |
| Status | active |
| Status summary | Triage verdict for the 4 hits the CM-DANGEROUS-COMBINATION-FAIL-CLOSED gate detects but never sees (repo root + tools/ absent from DANGER_ROOTS). Verdict: 4/4 FALSE POSITIVE for §11.4.252. Two REAL defects found elsewhere in the same file, invisible to this gate. |

## Instrument + provenance

- Gate: `constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh`
- sha256 **before** use: `099d412b39ea8d72e345365fd7954e6edfdf430b3672ec7bceaf0f00f56282a5`
- sha256 **after** use: `099d412b39ea8d72e345365fd7954e6edfdf430b3672ec7bceaf0f00f56282a5`
- **UNCHANGED** — BOB-195 r7 did not move the gate during this run. Gate was run UNMODIFIED, against scratch copies. No production source was edited.

### Control needle (§11.4.201(7)(b)) — instrument proven in BOTH directions

Isolated control arm (`scratchpad/bob205_triage/ctrl/`, fresh dir — the shared `scratchpad/needle/` already held another session's fixtures and was NOT used):

| Fixture | Shape | Expected | Observed |
|---|---|---|---|
| `needle_positive.py` | `except Exception: pass` | FIRE | **FIRED** (1 hit, line 4) |
| `negctrl_clean.py` | `except ... : log(); raise` | SILENT | **SILENT** (0 hits) |

Instrument can see through this exact path, and is not a blanket flagger. Therefore the ARM-2 result below is evidence, not a null.

### ARM 2 — the four hits, reproduced verbatim

All four report the **same** detector class — shape **(A2) SILENT DEFAULT** ("exception handler returns a trivial literal with no re-raise/log"). None is (A1) `pass`; none is (C) `contextlib.suppress`.

```
webui-bridge.py:295
tools/plugin_update_automation.py:189
tools/plugin_update_automation.py:198
tools/plugin_update_automation.py:215
→ 4 fail-open anti-pattern hit(s) found (§11.4.252), rc=1
```

That every hit is A2 is the load-bearing structural fact: A2 is precisely the shape whose documented sibling false positive (BOB-189) is *"a `return False` inside a validator IS the refusal."* The gate classifies by local syntactic shape and never consults the caller — by its own honest scope note it does not attempt to prove the ≥2-capability combination §11.4.252 actually requires.

---

# Hit 1 — `webui-bridge.py:295` — FALSE POSITIVE

```
280     def _is_root_liveness_probe(self):
...
293         try:
294             parsed = urllib.parse.urlparse(self.path)
295         except Exception:
296             return False
```

```
297         if parsed.path != "/":
298             return False
```

**Call site (the discriminator):**

```
427         except urllib.error.URLError as e:
435             if self._is_root_liveness_probe():
436                 self._send_root_liveness(e.reason)      #
        lenient: HTTP 200
437             else:
438                 self.send_error(502, ...)               #
        strict: HTTP 502
```

`False` selects the **502** branch. The `except` therefore *denies* the lenient path on an unparseable request path. The refusal direction is **fail-CLOSED** — flagging it inverts the meaning, exactly the BOB-189 shape.

**Capabilities combined: ZERO of the six.**
`_is_root_liveness_probe` is a pure classifier over `self.path` and `self.command`: no mutation, no credential access, no external call, no exec, nothing irreversible. §11.4.252 binds paths combining ≥2. This hit fails the capability test *and* the direction test.

Note the two remaining `return False` lines at 298/300 are the identical predicate-refusal shape and are not flagged only because they sit outside a handler — confirming the flag tracks syntax position, not semantics.

# Hit 2 — `tools/plugin_update_automation.py:189` — FALSE POSITIVE

```
183     def _download_url(self, url: str, timeout: int = 30) ->
        str | None:
185         try:
186             req = Request(url, headers={"User-Agent":
        "Mozilla/5.0"})
187             with urlopen(req, timeout=timeout) as response:
188                 return response.read().decode("utf-8")
189         except (URLError, HTTPError):
190             return None
```

**Call site:**

```
108                 upstream_content = self._download_url(url)
110                 if upstream_content is None:
111                     print_warning(f"{plugin_name}: Could not
        fetch upstream")
112                     continue
```

`None` is an **explicitly handled** refusal: the operator is warned and the plugin is **skipped** — no write occurs. The handler is also **bounded** to `(URLError, HTTPError)`, a declared tolerance; anything else propagates to the outer `except Exception` at :134 which prints an error. Fail-closed, twice over.

## Hit 3 — `tools/plugin_update_automation.py:198` — FALSE POSITIVE for §11.4.252 (separate minor defect noted)

```
192     def _extract_version(self, filepath: str) -> str:
194         try:
195             with open(filepath, encoding="utf-8") as f:
196                 content = f.read()
197             return self._extract_version_from_content(content)
198         except:
199             return "unknown"
```

`local_version` flows ONLY into the human print at :120 and the report dict at :125. The update decision is made at :116 by **hash comparison** (`local_hash != upstream_hash`), and `main()` at :250 branches on `update["status"]` — never on a version string. `"unknown"` therefore gates nothing: no capability is combined on this path, and no fail-open exists.

**Separate real (minor) defect, NOT §11.4.252:** the handler is a **bare `except:`**, which catches `BaseException` — so a `KeyboardInterrupt` landing inside `_extract_version` during a `--check` sweep over 14 upstream URLs is swallowed and the loop continues to the next plugin. Operator-visible symptom: Ctrl-C appears not to work. Code-quality/signal-handling class, not fail-open.

## Hit 4 — `tools/plugin_update_automation.py:215` — FALSE POSITIVE

```
210     def _validate_plugin(self, content: str) -> bool:
212         try:
213             compile(content, "<string>", "exec")
214             return True
215         except SyntaxError:
216             return False
```

**Call site:**

```
150         try:
151             if not self._validate_plugin(upstream_content):
152                 result["status"] = "failed"
153                 result["error"] = "Validation failed"
154                 return result           # ← returns BEFORE the
        write at :165
```

This `return False` **is the refusal**. It aborts the update before the `open(local_path, "w")` at :165. The handler is bounded to `SyntaxError`. The canonical BOB-189 false positive.

---

# Findings this gate CANNOT see (unanticipated)

## REAL-A — `update_plugin` is a genuine §11.4.252 dangerous combination, unflagged

`update_plugin` (:139–176) combines **three** capabilities:

- **untrusted input** — `upstream_content` fetched over the network from `UPSTREAM_SOURCES` (:67–82): 14 URLs across **four** GitHub repos, three of which are third-party personal repos, not the official `qbittorrent` org (`LightDestory/...`, `MadeOfMagicAndWires/...`).
- **mutation** — written to `plugins/<name>.py` (:165–166).
- **deferred code-exec** — those files are the qBittorrent nova3 search-plugin engines; `install-plugin.sh` copies them into `config/qBittorrent/nova3/engines/`, where qBittorrent **executes** them.

The only gate on that path is `_validate_plugin` → `compile(content, "<string>", "exec")`, a **syntax** check. It proves the bytes parse as Python; it proves nothing about what they do. There is no signature check, no pinned commit/tag, no content hash allowlist, no diff review. A changed or compromised upstream yields arbitrary Python written to a path that later executes.

The gate does not flag this because it only detects swallowed-exception shapes. **Net: in this file the gate produced 3 false positives and missed the one real dangerous combination** — the §11.4.201 both-directions failure in one file.

### REAL-B — documented rollback does not exist; a failed write leaves a truncated plugin

`tools/README.md` states: *"The script validates each downloaded plugin with `python3 -m py_compile` before installing it … **If validation fails the backup is restored.**"* Both halves are false against the code:

- Validation is in-process `compile()` (:213), not `py_compile`.
- **No restore path exists.** `shutil.copy2` appears exactly once, at :227, copying *forward* (source → backup). Grep proven seeing (`shutil` import :22, `backup` at :90/:162/:218–228); reverse-direction copy: zero occurrences.

Consequence: :165 opens with mode `"w"`, which **truncates immediately**. A failure mid-write (disk full, interrupt, decode error) leaves the plugin file truncated; the handler at :171–174 records `status="failed"` and prints `"Update failed"` — and the backup created at :162 is never restored.

**User-observable harm:** the operator is told the update **failed**, and concludes the previous plugin is intact — while a **truncated, non-importable** plugin sits at `plugins/<name>.py`. The next `./install-plugin.sh` copies that corrupt file into the engines directory and the search engine silently stops working. The `.bak` needed to recover exists but the tool never mentions or uses it. README also claims JSON reports go to stdout; :274 writes them to `args.output` (default `plugin_update_report.json`).

Severity: **Low-to-Medium**, bounded by REACHABILITY — see Q2: this script is invoked by nothing; it only runs when an operator types it by hand.

---

# Exposure fact for webui-bridge.py severity (established, not assumed)

```
447     server = ThreadingHTTPServer(("", BRIDGE_PORT),
        WebUIBridgeHandler)
74  BRIDGE_PORT = int(os.environ.get("BRIDGE_PORT", "7188"))
```

`("", PORT)` is `INADDR_ANY` — the bridge listens on **all interfaces**, not loopback. So :7188 is **LAN-reachable** (internet-reachable only if the operator forwards the port). It is neither localhost-only nor internet-exposed by default.

This raises the stakes for any *genuine* fail-open in this file — but Hit 1 is not one: it is a pure classifier whose refusal selects the stricter branch. No severity is assigned because no real defect was found at :295.

# Answers

**Q1 — one root cause or three?** Three hits, **one primitive**, but the primitive is in the DETECTOR, not the code (§11.4.250). All three tools/ hits — and the webui-bridge one — are the identical **A2 "silent default return"** shape, and in every case the returned literal is the function's *designed refusal value* consumed by a caller that checks it. The tools/ code has no shared fail-open primitive; what is shared is the gate's inability to read the call site. Fixing "three instances" in the code would mean damaging three correct refusals.

**Q2 — is `tools/` first-party production, or dev-only? Dev-only. It should be fenced out of the gate's scope — DECLARED, per §11.4.224(E), never silently.** Evidence: `tools/` contains exactly **2** tracked files (`README.md`, `plugin_update_automation.py`); neither is gitignored (`git check-ignore` rc=1 for both). The script is invoked by **nothing** — a control-needle-proven repo grep (91 hits for the known-present needle `install-plugin.sh`, 0 for the negative control) finds `plugin_update_automation` referenced only by its own README, its own docstring, and the BOB-205 note in `docs/Issues.md`; zero hits across `*.sh`/`*.yml`/`*.yaml`/`*.py`/`Makefile` outside `tools/`. Its own README self-declares: *"Standalone developer scripts that do **not** run inside a container and are not invoked by the normal start/stop flow."* Last functional commit `01c69f9` 2026-04-12; only later touch was the repo-wide CI sweep `4e855e0` 2026-04-23. Recommended exclusion class: **non-shipping developer utility** — it ships in no container, no image, no runtime path. Caveat: it is first-party, so §11.4.224(E) requires the exclusion carry a tracked §11.4.197 item, and REAL-A/REAL-B above should be tracked regardless of scope, since an operator running `--update` by hand still executes them.

**Q3 — is `webui-bridge.py` live or superseded? LIVE — decisively.** 142 tracked files reference `webui-bridge`, including `config/served_ports.yaml` and `challenges/security/credential_leak_audit.sh`. `download-proxy` actively probes it: `bridge_health()` exists at `download-proxy/src/api/__init__.py:198`, matching the docstring at :283–285. Its most recent commit is a real functional fix — `6e3da73 fix(bridge): target host-reachable qBittorrent (:7186)`, 2026-06-14, the newest of 12 commits. Critically, `config/served_ports.yaml` :66–69 records that the Go `webui-bridge` is *a separate binary the `qbittorrent-proxy-go` container never starts*, so **nothing else binds 7188**: the Python file is the only thing that actually serves the port in the default deployment. It is not superseded, so §11.4.124 does not apply and no removal question arises. Its absence from `DANGER_ROOTS` is a real scope gap — a live, LAN-bound HTTP service handling request paths, outbound calls, and environment credentials should be IN scope even though its one current hit is a false positive.